

ADR-07 — Protección de rutas mediante Auth Guards en Angular

Estado

Aceptado

Contexto

El sistema **ERP Iglesias** utiliza **Angular** como frontend para interactuar con la API del backend.

En aplicaciones web modernas es necesario controlar el acceso a determinadas rutas de la aplicación, especialmente aquellas que contienen información sensible o funcionalidades del sistema.

Por ejemplo, rutas como:

```
/dashboard  
/payments  
/courses
```

deben ser accesibles únicamente por usuarios que hayan iniciado sesión en el sistema.

Sin un mecanismo de control de acceso en el frontend, cualquier usuario podría intentar navegar directamente a estas rutas mediante la URL.

Problema

Si las rutas del sistema no están protegidas:

- usuarios no autenticados podrían acceder a funcionalidades del sistema
- se expone información sensible de la aplicación
- la navegación del sistema no tendría control de acceso
- se compromete la seguridad del frontend

Esto genera **riesgos de seguridad y acceso no autorizado a funcionalidades del sistema**.

Decisión Arquitectónica

Se implementa **Auth Guards en Angular** para proteger las rutas del sistema.

Los **Guards** permiten interceptar la navegación antes de que una ruta sea cargada y verificar si el usuario tiene permisos para acceder.

Arquitectura utilizada:

```
Usuario intenta acceder a ruta
↓
Angular Router
↓
AuthGuard
↓
Verificación de autenticación
↓
Acceso permitido o redirección
```

Patrón de Diseño Aplicado

Guard Pattern

Este patrón permite proteger el acceso a determinados recursos verificando condiciones antes de permitir la ejecución de una acción.

Principio SOLID Aplicado

Single Responsibility Principle (SRP)

Cada componente del sistema tiene una responsabilidad específica:

- Router → gestionar navegación entre rutas
- AuthGuard → verificar autenticación del usuario
- AuthService → gestionar estado de autenticación

Implementación Actual

El proyecto ya cuenta con un **Auth Guard funcional** que controla el acceso a las rutas protegidas.

Archivo implementado:

```
auth.guard.ts
```

Ejemplo de implementación actual:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn) {
    return true;
  }

  return router.parseUrl('/login');
};
```

Este guard consulta el estado de autenticación del usuario mediante **AuthService**.

Configuración en el Router

Las rutas protegidas utilizan el guard para controlar el acceso.

Ejemplo de configuración:

```
{
  path: 'dashboard',
  component: DashboardComponent,
  canActivate: [authGuard]
}
```

Ejemplo para Entenderlo

Flujo de navegación:

```
Usuario intenta entrar a /dashboard
↓
authGuard verifica autenticación
↓
Si está autenticado → acceso permitido
↓
Si no está autenticado → redirige a /login
```

Beneficios Arquitectónicos

La implementación de Auth Guards proporciona:

- control de acceso a rutas del sistema
- mejora de la seguridad en el frontend
- prevención de accesos no autorizados
- mejor organización de la lógica de autenticación
- experiencia de usuario más controlada en la navegación

Impacto en el Sistema

La implementación de guards no modifica la funcionalidad del sistema para los usuarios autenticados.

Sin embargo, mejora la seguridad del sistema al evitar accesos no autorizados a rutas protegidas.

Resultado Arquitectónico

Arquitectura aplicada en el frontend:

```
Usuario
↓
Angular Router
↓
AuthGuard
↓
AuthService
↓
Acceso permitido o redirección
```

Esta decisión permite implementar un control de acceso claro y seguro dentro del frontend de la aplicación.